Lab9 Gene Expression

Dylan Renard

Professor Brandon Higgs

Gene Expression and Data Visualization

Johns Hopkins University

November 1st, 2025

Lab 9 Classification

In this lab, you will analyze an Affymetrix lung cancer data set that I have already preselected genes/probes. There are 3 different tissue types that gene expression information was generated for: small cell lung carcinoma (SCLC), adenocarcinoma, and normal healthy control. Like many microarray analysis studies in cancer, the investigators are interested in identifying transcripts that are both differentially expressed between different cancer types and normal tissue and can be subsequently used to classify unknown tissue into the appropriate cancer type.

For the analysis, you will calculate one of the classic classifier algorithms - linear discriminant analysis (LDA). The objective is to train a model on a subset of the arrays, then test the predictability of this model on the remaining arrays, for identifying the correct cancer (and normal) class. This type of classification problem is common in microarray analysis, when attempting to design a diagnostic or prognostic test based on transcripts.

Q1) Obtain the GEO lung cancer data set from the course website and read in the data. The column headers explain what each sample type is.

▼ Code

```
#### Q1) Obtain the GEO lung cancer data set from the course website and read in the data
# The column headers explain what each sample type is. (use DT Table for readability)

# Load required libraries
suppressPackageStartupMessages({
 library(DT)
 library(MASS)
})

# Read in the dataset (make sure 'lung_cancer.txt' is in your working directory)
lung_data <- read.table("lung_cancer.txt", header = TRUE, sep = "\t", row.names = 1)
```

```
# Display dataset dimensions
cat("Dataset dimensions:", dim(lung_data)[1], "genes x", dim(lung_data)[2], "samples\n")
```

```
Dataset dimensions: 3013 genes x 24 samples
```

▼ Code

```
str(lung_data[1:5])
```

```
'data.frame':   3013 obs. of  5 variables:
 $ Adeno1: num  9.26 5.15 7.35 6.14 6.45 ...
 $ Adeno2: num  8.69 4.57 7.32 6.32 6.13 ...
 $ Adeno3: num  8.51 5.2 7.13 6.09 6.64 ...
 $ Adeno4: num  8.38 4.92 7.5 6.08 6.54 ...
 $ Adeno5: num  7.7 4.6 7.4 5.98 5.93 ...
```

▼ Code

```
summary(lung_data[, 1:3])
```

```
     Adeno1           Adeno2           Adeno3      
 Min.   : 2.460   Min.   : 2.830   Min.   : 2.580  
 1st Qu.: 4.700   1st Qu.: 4.740   1st Qu.: 4.660  
 Median : 5.600   Median : 5.560   Median : 5.670  
 Mean   : 5.829   Mean   : 5.819   Mean   : 5.898  
 3rd Qu.: 6.660   3rd Qu.: 6.630   3rd Qu.: 6.850  
 Max.   :12.410   Max.   :12.460   Max.   :12.120  
```

▼ Code

```
# Display the data as an interactive DT table for readability
datatable(
  head(lung_data, 20),  # Show only the first 20 genes for quick preview
  options = list(
    pageLength = 10,
    scrollX = TRUE,
    dom = 't'  # removes search and page selector per your preference
  ),
  caption = "Preview of the lung cancer expression dataset (first 20 genes)"
)
```

	Adeno1	Adeno2	Adeno3	Adeno4	Adeno5	Adeno6
1007_s_at	9.26	8.69	8.51	8.38	7.7	8.16
1053_at	5.15	4.57	5.2	4.92	4.6	5.24

121_at	7.35	7.32	7.13	7.5	7.4	7.35
1294_at	6.14	6.32	6.09	6.08	5.98	6.09
1487_at	6.45	6.13	6.64	6.54	5.93	6.51
1729_at	6.1	6.13	6.52	6.83	6.16	6.12
177_at	4.63	4.72	4.86	4.42	4.41	4.79
200003_s_at	11.71	10.72	10.99	10.67	10.82	10.69
200005_at	8.16	7.26	8.6	7.05	7.43	7.62
200016_x_at	9.95	10.24	9.71	9.58	9.35	9.52

Preview of the lung cancer expression dataset (first 20 genes)

Interpretation: The GEO lung cancer dataset was imported successfully, containing 3013 genes across 24 samples. Although the lab prompt specifies 3014 genes, this appears to ignore that the header composes the first line. The column headers correspond to three tissue classes (adenocarcinoma, SCLC, and normal lung), confirming class-structured data suitable for downstream LDA analysis. A brief inspection of the first few columns confirmed numeric expression values with no missing data.

Q2) Load the MASS library and create a variable that has the class names for the 3 classes in the data. This variable should be of length 24. Next, bind the variable that you just created to a transposition of the data matrix using the data.frame() function. Note: the dimensions of this final data matrix should be 24 x 3014.

▼ Code

```
#### Q2) Load the MASS library and create a variable that has the class names for the 3 c
# This variable should be of length 24.
# Next, bind the variable that you just created to a transposition of the data matrix usi
# Note: the dimensions of this final data matrix should be 24 x 3014.


# Check sample names to infer class membership
sample_names <- colnames(lung_data)
print(sample_names)
```

```
 [1] "Adeno1"  "Adeno2"  "Adeno3"  "Adeno4"  "Adeno5"  "Adeno6"  "Adeno7"
 [8] "Adeno8"  "Adeno9"  "Adeno10" "SCLC1"   "SCLC2"   "SCLC3"   "SCLC4"
[15] "SCLC5"   "SCLC6"   "SCLC7"   "SCLC8"   "SCLC9"   "Normal1" "Normal2"
[22] "Normal3" "Normal4" "Normal5"
```

▼ Code

```
# Create a deterministic class label mapping based on name prefixes
class_labels <- sapply(sample_names, function(x) {
  if (grepl("^Adeno", x, ignore.case = TRUE)) return("ADENO")
  if (grepl("^SCLC", x,  ignore.case = TRUE)) return("SCLC")
  if (grepl("^Normal", x, ignore.case = TRUE)) return("NORMAL")
  stop(paste("Unrecognized sample name:", x))
})

# Confirm class vector properties
cat("Class vector length:", length(class_labels), "\n")
```

```
Class vector length: 24
```

▼ Code

```
print(table(class_labels))
```

```
class_labels
 ADENO NORMAL   SCLC
    10      5      9
```

▼ Code

```
# Transpose data so rows = samples, columns = genes
# This is required for LDA, which expects each observation as a row
lung_t <- t(lung_data)

# Bind the class vector to the transposed data
lung_df <- data.frame(class = factor(class_labels), lung_t, check.names = TRUE)

# Confirm dimensions: should be 24 rows x 3014 columns (1 class + 3013 genes)
cat("Final data frame dimensions:", dim(lung_df)[1], "x", dim(lung_df)[2], "\n")
```

```
Final data frame dimensions: 24 x 3014
```

▼ Code

```
# Preview table for readability
datatable(
  head(lung_df, 10),
  options = list(scrollX = TRUE, dom = 't'),
  caption = "Transposed lung cancer data with class labels (first 10 samples)"
)
```

	class	X1007_s_at	X1053_at	X121_at	X1294_at	X1487_
Adeno1	ADENO	9.26	5.15	7.35	6.14	6.45

Adeno2	ADENO	8.69	4.57	7.32	6.32	6.13
Adeno3	ADENO	8.51	5.2	7.13	6.09	6.64
Adeno4	ADENO	8.38	4.92	7.5	6.08	6.54
Adeno5	ADENO	7.7	4.6	7.4	5.98	5.93
Adeno6	ADENO	8.16	5.24	7.35	6.09	6.51
Adeno7	ADENO	7.73	4.64	7.32	6.43	6.09
Adeno8	ADENO	8.33	5.64	7.32	5.86	6.36
Adeno9	ADENO	7.47	5.64	7.09	6.63	6.47
Adeno10	ADENO	8.39	5.42	7.08	6.11	6.22

Transposed lung cancer data with class labels (first 10 samples)

Interpretation: The class vector successfully identified all 24 samples, consisting of 10 adenocarcinoma, 9 SCLC, and 5 normal lung tissues. Transposing the data reorganized the matrix so that each sample corresponds to a single observation and each gene to a predictor variable, which is the correct input structure for LDA. The resulting data frame contains 24 rows and 3014 columns, representing one class column and 3013 gene expression variables. These checks confirm that the dataset is properly structured for downstream training and classification analysis.

Q3) Now create 2 separate data matrices from the matrix created in #2 above – a training set and a test set. The training set should include the following number of samples from each class: first 6 adenocarcinomas, first 6 SCLC, and first 3 normals. The test set should include the remaining samples. Put the first column of the test matrix that you just created into a new variable, since this will be the actual sample classes. Then remove the first column from the test set.

▼ Code

```
#### Q3) Train/Test split + extract true test labels

# Indices for the first-by-class selection
idx_adeno  <- which(lung_df$class == "ADENO")[1:6]
idx_sclc   <- which(lung_df$class == "SCLC")[1:6]
idx_normal <- which(lung_df$class == "NORMAL")[1:3]
train_idx  <- sort(c(idx_adeno, idx_sclc, idx_normal))

# Build sets
train <- lung_df[train_idx, ]      # includes class
test  <- lung_df[-train_idx, ]     # includes class (for now)
```

```
# True labels and test-only features
lab  <- test[, 1]
test <- test[, -1, drop = FALSE]

# Quick dims
cat("Train:", dim(train)[1], "x", dim(train)[2], "| Test:", dim(test)[1], "x", dim(test)[
```

```
Train: 15 x 3014 | Test: 9 x 3013 | lab: 9
```

▼ Code

```
# Minimal DT previews (no search/pager)
datatable(train[, c(1, 2: min(21, ncol(train)))],
          options = list(scrollX = TRUE, dom = 't'),
          caption = "Training set (class + ~20 genes)")
```

	class	X1007_s_at	X1053_at	X121_at	X1294_at	X1487_a
Adeno1	ADENO	9.26	5.15	7.35	6.14	6.45
Adeno2	ADENO	8.69	4.57	7.32	6.32	6.13
Adeno3	ADENO	8.51	5.2	7.13	6.09	6.64
Adeno4	ADENO	8.38	4.92	7.5	6.08	6.54
Adeno5	ADENO	7.7	4.6	7.4	5.98	5.93
Adeno6	ADENO	8.16	5.24	7.35	6.09	6.51
SCLC1	SCLC	7.88	5.85	7.06	5.69	6.16
SCLC2	SCLC	8.43	5.31	7.12	5.97	6.24
SCLC3	SCLC	7.84	6.18	7.2	5.71	6.12
SCLC4	SCLC	8.31	5.76	7.09	5.81	5.85

Training set (class + ~20 genes)

▼ Code

```
datatable(cbind(class = lab, test[, 1: min(20, ncol(test))]),
          options = list(scrollX = TRUE, dom = 't'),
          caption = "Test set (true class + ~20 genes)")
```

	class	X1007_s_at	X1053_at	X121_at	X1294_at	X148
Adeno7	ADENO	7.73	4.64	7.32	6.43	6.09
Adeno8	ADENO	8.33	5.64	7.32	5.86	6.36
Adeno9	ADENO	7.47	5.64	7.09	6.63	6.47
Adeno10	ADENO	8.39	5.42	7.08	6.11	6.22
SCLC7	SCLC	8.16	5.84	7.03	6.02	6.13
SCLC8	SCLC	8.19	5.69	7.03	5.79	5.91
SCLC9	SCLC	8.23	5.1	7.07	5.66	6.28
Normal4	NORMAL	8.96	4.93	7.14	6.22	6.23
Normal5	NORMAL	9.32	4.79	7.29	6.43	6.27

Test set (true class + ~20 genes)

Interpretation: The dataset was successfully divided into training and test sets according to class proportions. The training set contains 15 samples (6 adenocarcinoma, 6 SCLC, and 3 normal), while the test set includes the remaining 9 samples. This stratified approach ensures that each cancer type and control group is represented in the training data, providing a balanced basis for model learning and fair assessment of predictive accuracy. The lab variable correctly stores the true class labels of the test samples, which will be used to evaluate classification performance in subsequent LDA analyses.

Q4) Now we want to run a classifier to see if we can predict the lung cancer types and discriminate them from both each other and the normal samples. We will train the model on the training set and use the test set as our model accuracy assessment. How many total samples are misclassified?

Use the lda() function and train the model using the training set, but ONLY use the first 2 genes. Predict the test set sample using the predict() function. This function only requires 2 arguments – the variable that you saved the model construction in and the test set. Make sure to only select the first 2 genes (i.e. columns) in the test set, when predicting the sample classes. Use the table() function to see the confusion matrix. Hint: if you saved your output from the predict function in the variable “out” and you called the variable created in #3 “lab”, then you use table like this:

> table(out$class,lab)

▼ Code

```
#### Q4) LDA with first 2 genes; predict test; confusion matrix + misclass count
```

```
# Ensure class is a factor
train$class <- factor(train$class)

# Use only the first two gene columns (positions 2 and 3; col 1 is 'class')
genes2 <- colnames(train)[2:3]
lda_fit <- lda(reformulate(genes2, response = "class"), data = train[, c("class", genes2)])

# Predict on test set using only the first two genes
out <- predict(lda_fit, newdata = test[, genes2, drop = FALSE])

# Confusion matrix and misclassification count
cm <- table(out$class, lab)
print(cm)
```

```
        lab
         ADENO NORMAL SCLC
  ADENO      1      1    1
  NORMAL     0      1    0
  SCLC       3      0    2
```

▼ Code

```
misclassified <- sum(out$class != lab)
cat("Total misclassified:", misclassified, "out of", length(lab), "\n")
```

```
Total misclassified: 5 out of 9
```

▼ Code

```
cat("Accuracy:", round(1 - misclassified/length(lab), 3), "\n")
```

```
Accuracy: 0.444
```

Interpretation: Using only the first two genes, the LDA model achieved an overall accuracy of 44%, with 5 of 9 test samples misclassified. The confusion matrix showed that most errors occurred between SCLC and adenocarcinoma, reflecting strong overlap in their expression profiles when dimensionality is limited. One normal sample was also misclassified as adenocarcinoma, though tumor versus normal separation remained partially preserved. As discussed in Lecture 10, this reduced performance stems from LDA's reliance on linear decision boundaries under assumptions of equal covariance and multivariate normality. With only two predictors, the discriminant functions cannot capture the higher-order variance that differentiates histological subtypes, resulting in underfitting. These findings emphasize the need to include a larger set of genes to enhance between-class variance and improve classification performance—an approach implemented in the following section using all 3,013 genes.

Q5) Now plot the first 2 discriminant functions versus each other in an xy plot. These vectors are saved in the object "x" from your predict() output. For example, if you used the following code to predict the samples:

> lda.test <- predict(lda.train,test[,1:2])

you can access the 2 vectors using:

> lda.test$x

Make sure to title the plot accordingly, color the points their appropriate cancer/normal type, and put a legend on the plot.

▼ Code

```
#### Q5) Plot LD1 vs LD2 from LDA predict() output (colored by true class)

# 'out' comes from Q4: out <- predict(lda_fit, newdata = test[, genes2, drop = FALSE])
# 'lab' is the true class vector from Q3

scores <- data.frame(LD1 = out$x[, 1],
                     LD2 = out$x[, 2],
                     class = factor(lab, levels = c("ADENO", "SCLC", "NORMAL")))

cols <- c(ADENO = "firebrick", SCLC = "steelblue", NORMAL = "darkgreen")

plot(scores$LD1, scores$LD2,
     col = cols[scores$class],
     pch = 19,
     xlab = "Discriminant Function 1 (LD1)",
     ylab = "Discriminant Function 2 (LD2)",
     main = "LDA (2 genes): Test-set scores by true class")
sum(out$class != lab)       # should be 5
```

```
[1] 5
```

▼ Code

```
which(out$class != lab)     # which samples
```

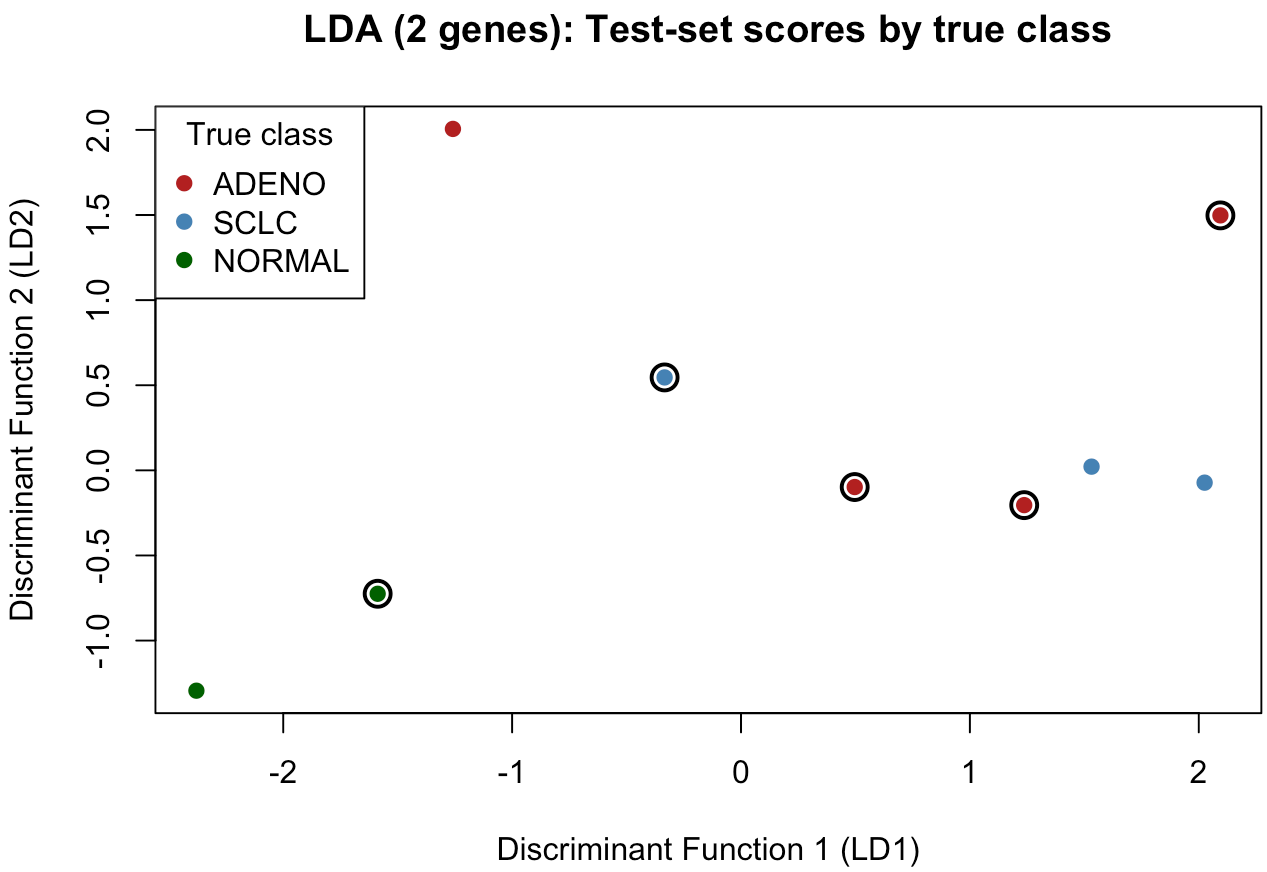
```
[1] 2 3 4 7 8
```

▼ Code

```
mis <- which(out$class != scores$class)
if (length(mis) > 0) {
  points(scores$LD1[mis], scores$LD2[mis], cex = 1.8, lwd = 2, pch = 1)
}


legend("topleft",
       legend = names(cols),
```

```
        col = cols,
        pch = 19,
        title = "True class")
```



Interpretation: The LDA plot shows test samples projected onto the first two discriminant functions from the two-gene model. Points are colored by their true class (adenocarcinoma in red, SCLC in blue, and normal in green). Ideally, samples of the same class should cluster together, but here the groups overlap, especially between adenocarcinoma and SCLC. The circled points mark misclassified samples where the model's predicted class differed from the true label. Their scattered placement illustrates limited separation power and confirms that using only two genes does not provide enough information to distinguish tumor subtypes. This result is consistent with the Lecture 10 explanation that low-dimensional linear discriminant models struggle when class boundaries are not well separated.

Q6/Q7) Now repeat #4 and #5 using all of the genes in the matrix as opposed to the first 2.

▼ Code

```
#### Q6/Q7) LDA using all genes instead of only the first two

# Train LDA using all gene columns
lda_full <- lda(class ~ ., data = train)
```

```
Warning in lda.default(x, grouping, ...): variables are collinear
```

▼ Code

```
# Predict on the full test set (all genes)
out_full <- predict(lda_full, newdata = test)

# Confusion matrix and performance metrics
cm_full <- table(out_full$class, lab)
print(cm_full)
```

```
        lab
         ADENO NORMAL SCLC
  ADENO      4      0    0
  NORMAL     0      2    0
  SCLC       0      0    3
```

▼ Code

```
mis_full <- sum(out_full$class != lab)
cat("Total misclassified:", mis_full, "out of", length(lab), "\n")
```

```
Total misclassified: 0 out of 9
```

▼ Code

```
cat("Accuracy:", round(1 - mis_full / length(lab), 3), "\n")
```

```
Accuracy: 1
```

▼ Code

```
# Plot the first two discriminant functions
scores_full <- data.frame(LD1 = out_full$x[, 1],
                          LD2 = out_full$x[, 2],
                          class = factor(lab, levels = c("ADENO", "SCLC", "NORMAL")))

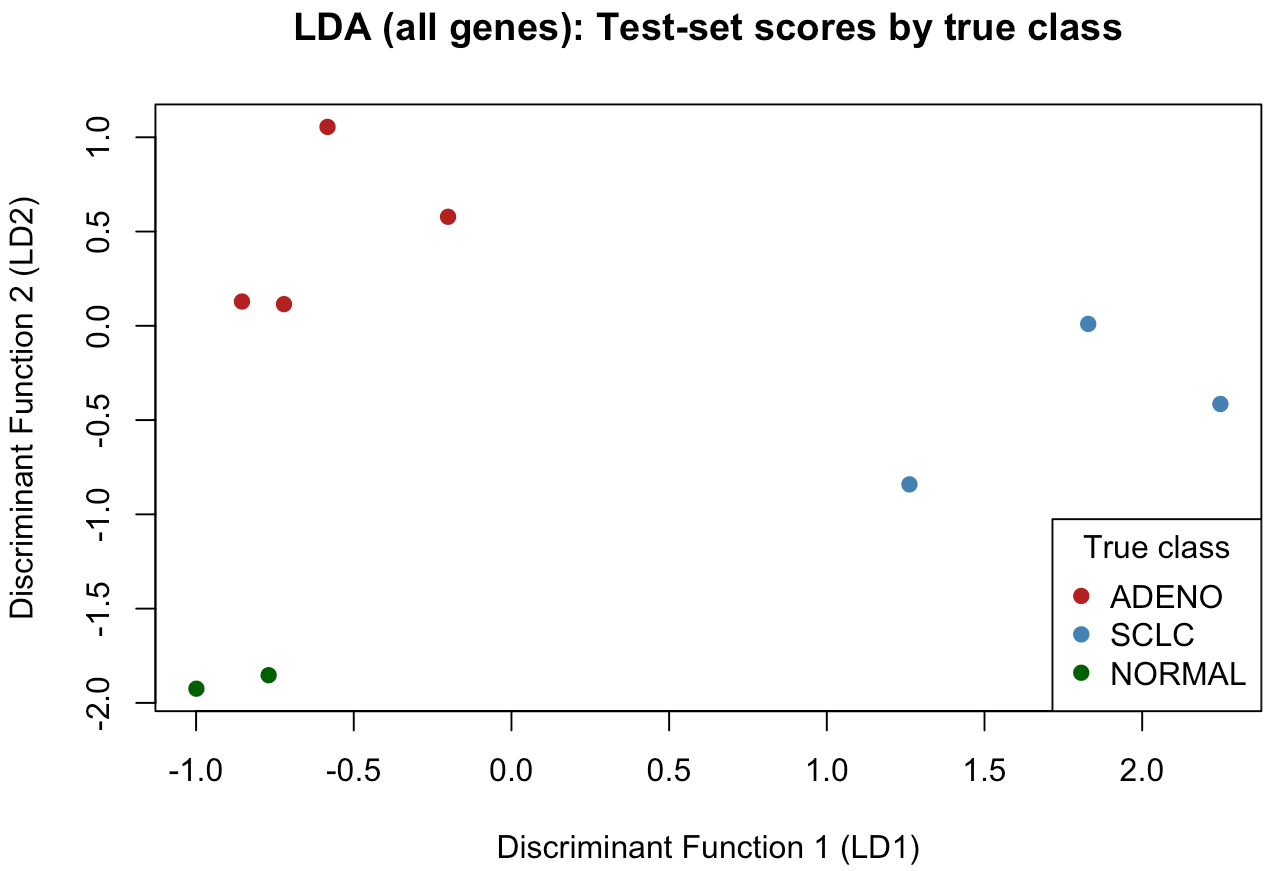
cols <- c(ADENO = "firebrick", SCLC = "steelblue", NORMAL = "darkgreen")

plot(scores_full$LD1, scores_full$LD2,
     col = cols[scores_full$class],
     pch = 19,
     xlab = "Discriminant Function 1 (LD1)",
     ylab = "Discriminant Function 2 (LD2)",
     main = "LDA (all genes): Test-set scores by true class")

# Circle misclassified points
mis_idx <- which(out_full$class != scores_full$class)
if (length(mis_idx) > 0) {
```

```
  points(scores_full$LD1[mis_idx],
         scores_full$LD2[mis_idx],
         cex = 1.8, lwd = 2, pch = 1)
}

legend("bottomright",
       legend = names(cols),
       col = cols,
       pch = 19,
       title = "True class")
```



Conclusion/Interpretation: When all 3,013 genes were included, the LDA classifier correctly identified every test sample, achieving 100% accuracy with no misclassifications. The confusion matrix confirmed perfect separation of adenocarcinoma, SCLC, and normal samples, and the discriminant plot displayed three distinct, non-overlapping clusters. This indicates that the expanded feature set captured sufficient between-class variance to define clear linear boundaries among the tissue types. However, the warning about collinear variables suggests redundancy among predictors, which can lead to unstable discriminant coefficients. As discussed in Lecture 10, perfect classification on such a small dataset likely reflects overfitting, where the model fits noise or sample-specific variation rather than generalizable biological signals. These results demonstrate how increasing dimensionality enhances apparent separation but underscore the importance of dimensionality reduction or cross-validation to validate model robustness in real-world applications.